

CYPHER

Training School on Scientific Machine Learning for Digital Twins

*System Identification and machine
learning for predicting reacting dynamics
Hands-on session*

Anh Khoa Doan

Department of Aeronautics

n.doan@imperial.ac.uk



Short instructions

Scripts and data available here:

https://github.com/ndoan-icl/CYPHER_MLSchool_Brussels2026

Search for ndoan-icl on google colab

Slides also available on the git repo

Alternatively, after copying the git repo:

Create an environment with:

```
conda create --name cypher --file requirements.txt
```

Short instructions

- Alternatively, install the following libraries (beyond the usual ones – numpy, scipy, matplotlib):
 - tensorflow
 - impulseest
 - scikit-learn
 - scikit-optimize
 - jupyterlab

Structure of the session

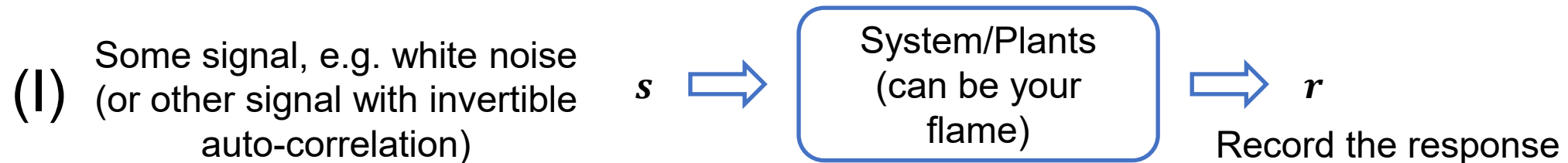
1. **5-minute recap**
2. Guide through example notebook: System Identification
3. Guide through example notebook: Lorenz system
4. Description of the exercise: Learning flame dynamics

Very short recap on System Identification

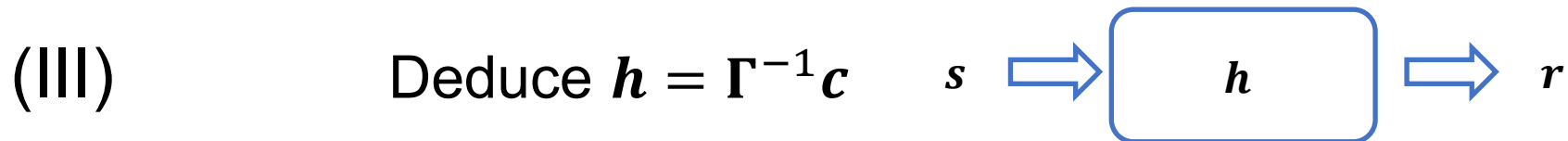
$$\rightarrow \mathbf{h} = \mathbf{\Gamma}^{-1} \mathbf{c} \text{ or } \mathbf{h}_{\alpha} = (\mathbf{\Gamma}^T \mathbf{\Gamma} + \alpha^2 \mathbf{R} \mathbf{R}^T)^{-1} \mathbf{\Gamma}^T \mathbf{c}$$

- If s is white noise, $\mathbf{\Gamma} = \mathbf{I}$ (identity matrix)

- Summary



(II) Compute $\mathbf{\Gamma}, \mathbf{c}$



(IV) With $\mathbf{h}$, estimate $\mathbf{r}$ for any s

Modelling of time series with feedforward neural networks

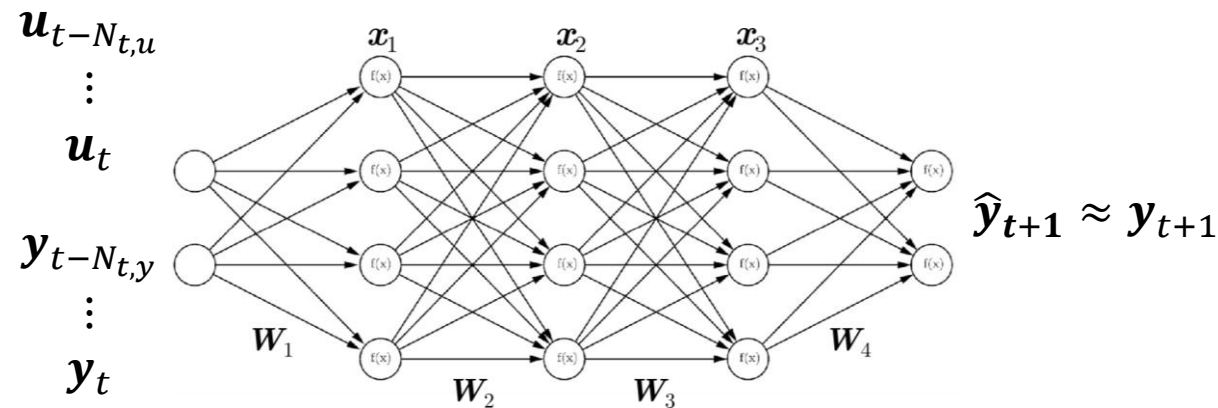
- Assume we want to model a time-dependent process:

$$\dot{\mathbf{y}}(t) = \mathcal{F}(\mathbf{y}, \mathbf{u})$$
$$\rightarrow \mathbf{y}_{t+1} = \mathcal{F}_d \left(\mathbf{y}_t, \dots, \mathbf{y}_{t-N_{t,y}}, \mathbf{u}_t, \dots, \mathbf{u}_{t-N_{t,u}} \right)$$

Typical feedforward-based architecture

Time series of past input
and prediction

$N_{t,u}, N_{t,y}$: duration of
“relevant past
information”



Modelling of time series with recurrent neural network

- Assume we want to model a time-dependent process:

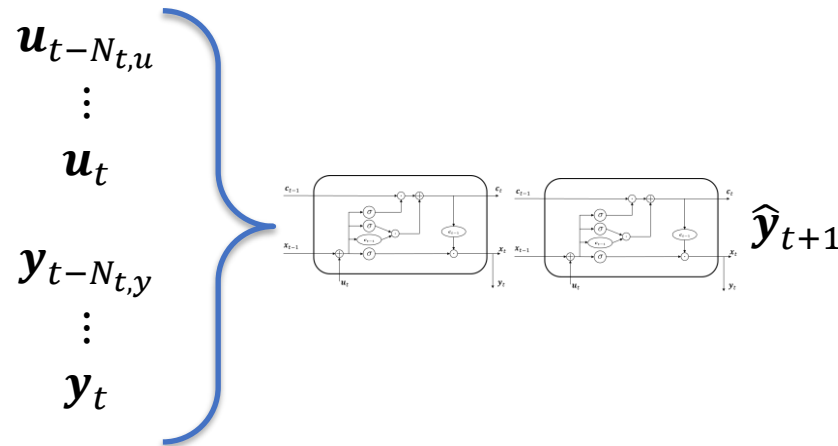
$$\dot{\mathbf{y}}(t) = \mathcal{F}(\mathbf{y}, \mathbf{u})$$

$$\rightarrow \mathbf{y}_{t+1} = \mathcal{F}_d \left(\mathbf{y}_t, \dots, \mathbf{y}_{t-N_{t,y}}, \mathbf{u}_t, \dots, \mathbf{u}_{t-N_{t,u}} \right)$$

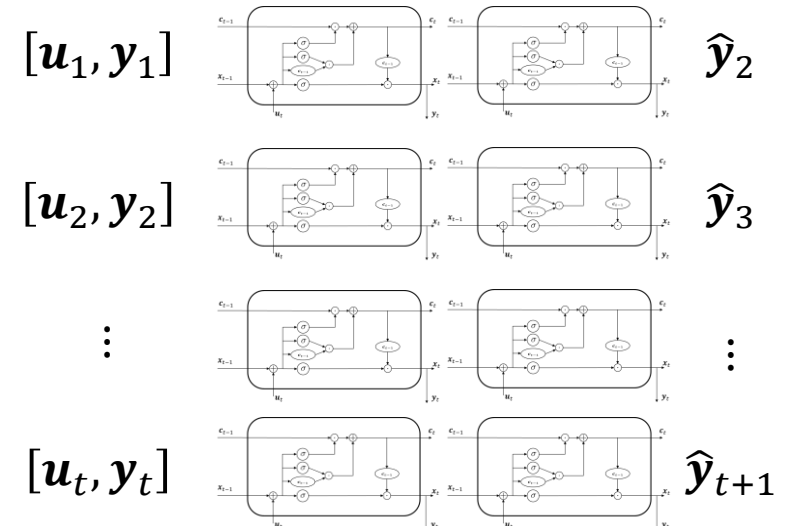
Two possible recurrent approaches with RNNs:

“Many to one” approach:

Time series of
past input and
prediction



“Many to many” approach:

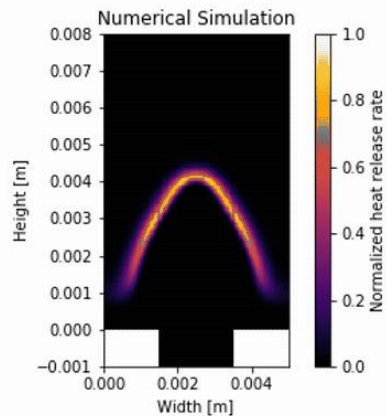


Structure of the session

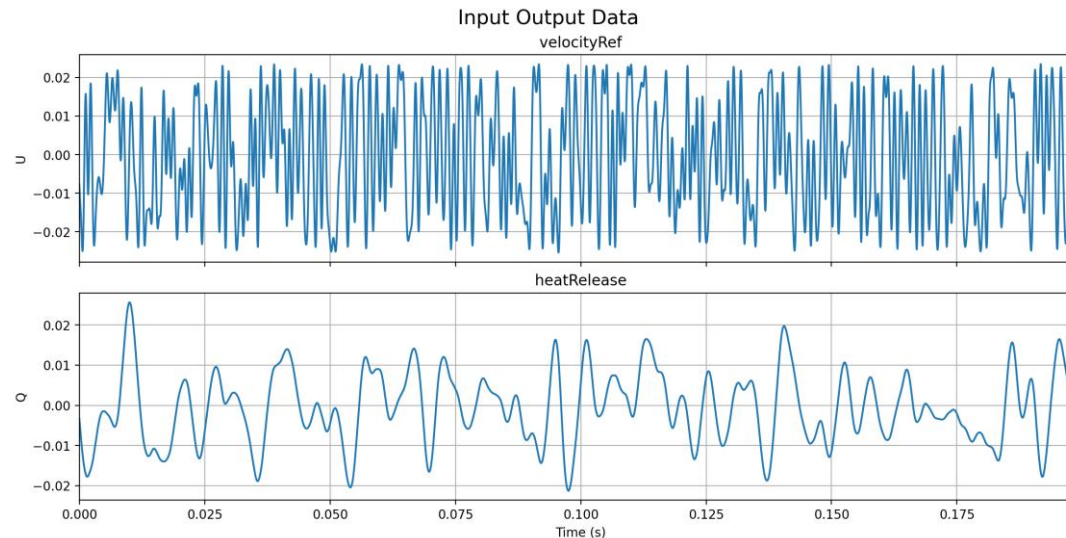
1. 5-minute recap
2. **Guide through example notebook: System Identification**
3. Guide through example notebook: Lorenz system
4. Description of the exercise: Learning flame dynamics

(Linear) Flame dynamics of a laminar slit flame

- Premixed methane-air laminar slit burner (equivalence ratio of 0.8)
- Fully resolved with DNS



- Time series of input (velocity fluctuation) and output (heat release rate fluctuation) provided



- Application of System Identification to obtain flame frequency response

Structure of the session

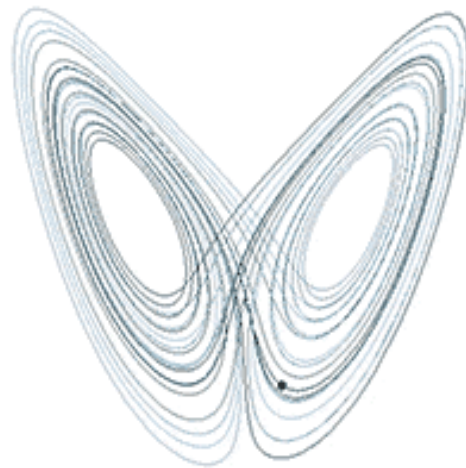
1. 5-minute recap
2. Guide through example notebook: System Identification
3. **Guide through example notebook: Lorenz system**
4. Description of the exercise: Learning flame dynamics

Guided exercise: Lorenz system

- Developed originally as a model of atmospheric convection

$$\frac{dx}{dt} = \sigma(y - x), \frac{dy}{dt} = x(\rho - z) - y, \frac{dz}{dt} = xy - \beta z$$

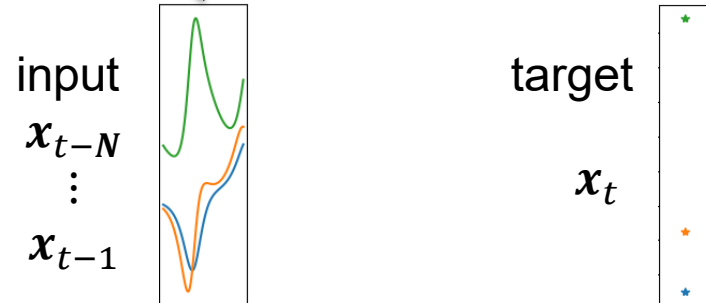
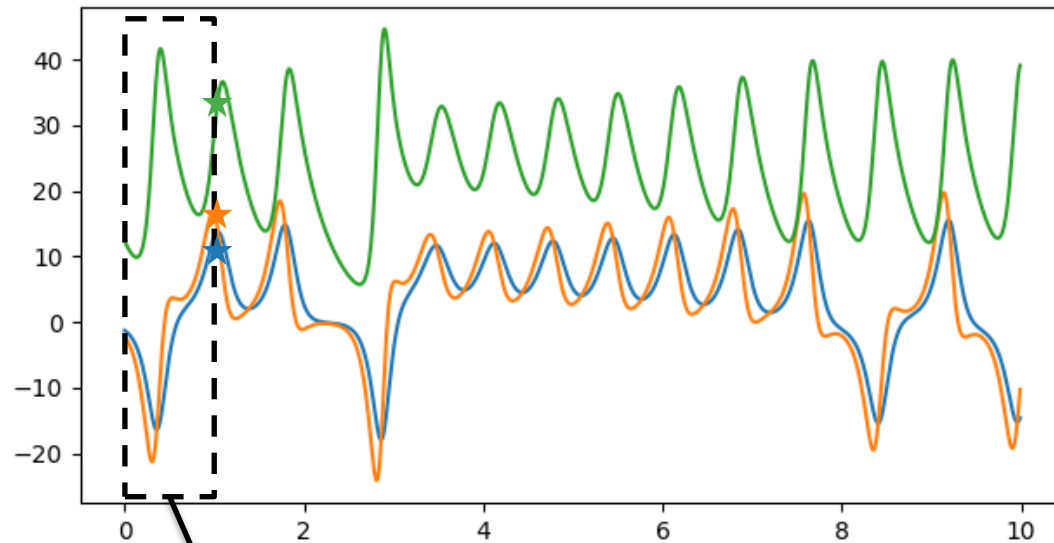
- Simplified model that exhibits chaotic behaviour.



$$\rho = 28, \sigma = 10, \beta = 8/3$$

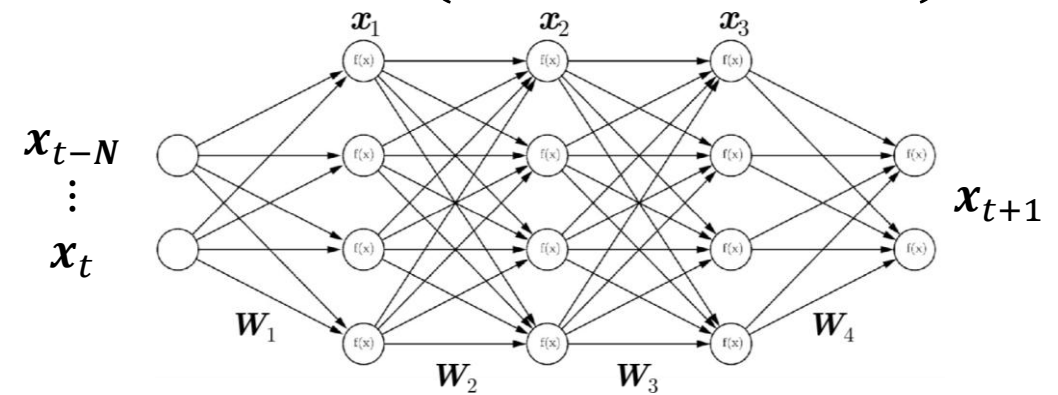
Guided exercise: Lorenz system

Data: Time series of the Lorenz system



Objective: predict next time step based on previous ones:

$$x(t+1) = \mathcal{F}(x(t), \dots, x(t-N))$$



Steps:

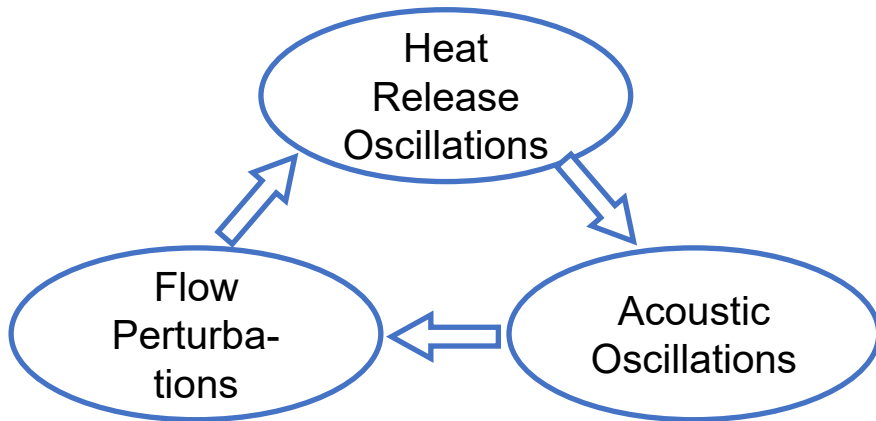
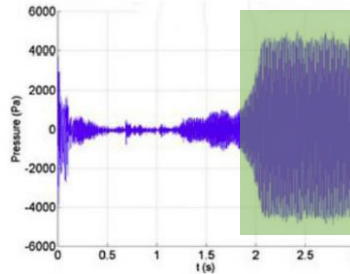
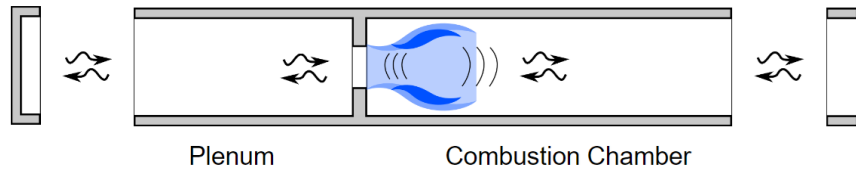
1. Restructure datasets into appropriate (input,pair)
2. Design the network
3. Train

Structure of the session

1. 5-minute recap
2. Guide through example notebook: System Identification
3. Guide through example notebook: Lorenz system
4. **Description of the exercise: Learning flame dynamics**

Thermoacoustic instabilities result from heat release rate oscillations

- Thermoacoustic instabilities: High amplitude pressure oscillations in 'lean premixed' combustors



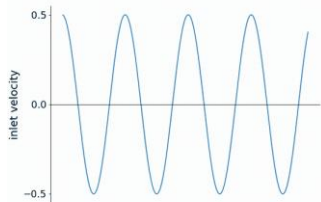
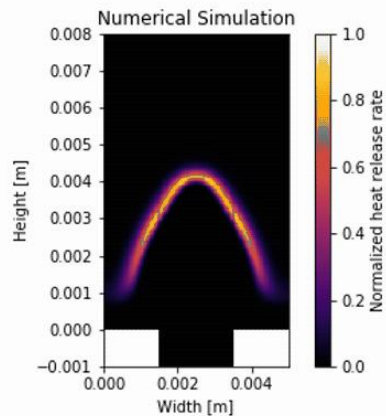
- Prediction requires a coupling between:

- *Acoustic model*

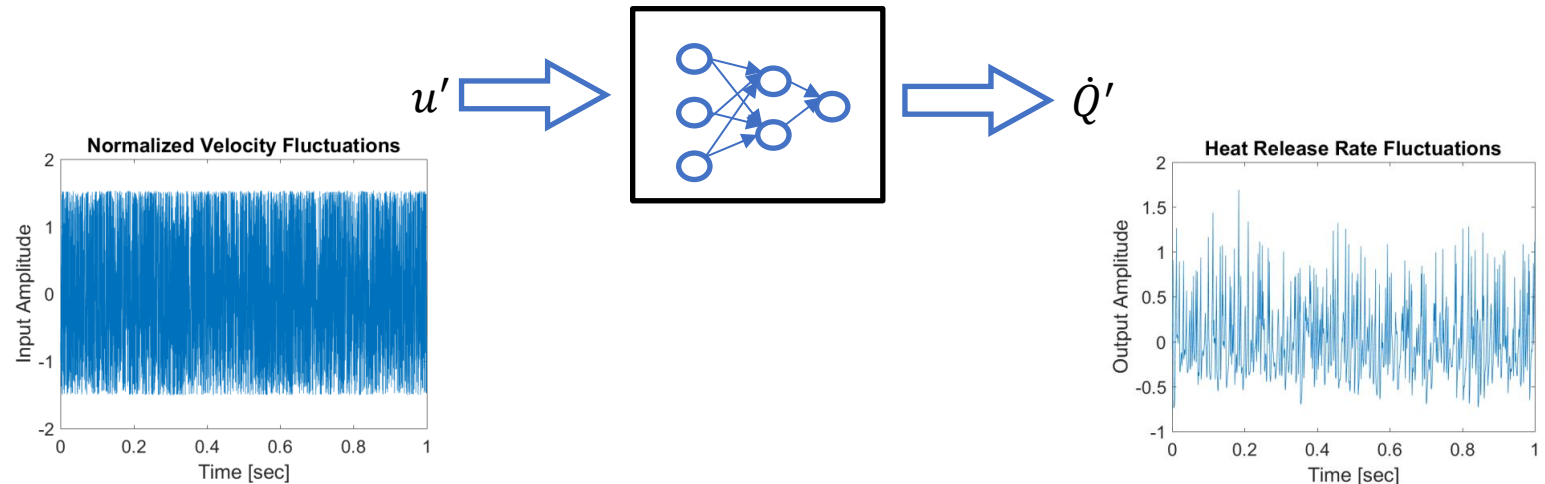
- *Flame model: $\dot{Q}' = f(u')$*

NN must be trained to learn flame dynamics ($\dot{Q}' = f(u')$) of a laminar slit flame

- Premixed methane-air laminar slit burner (equivalence ratio of 0.8)
- Fully resolved with DNS



Objective: Design a neural networks trained using the data from broadband simulation



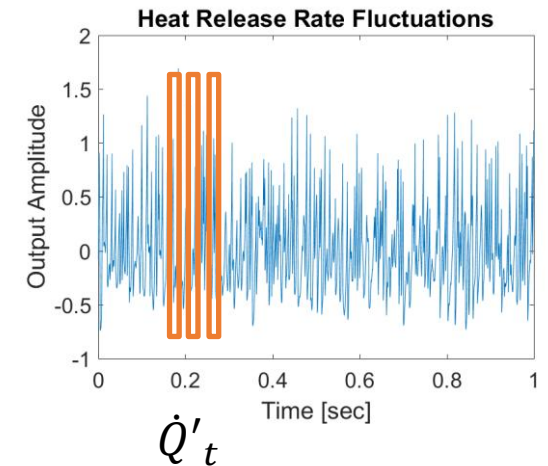
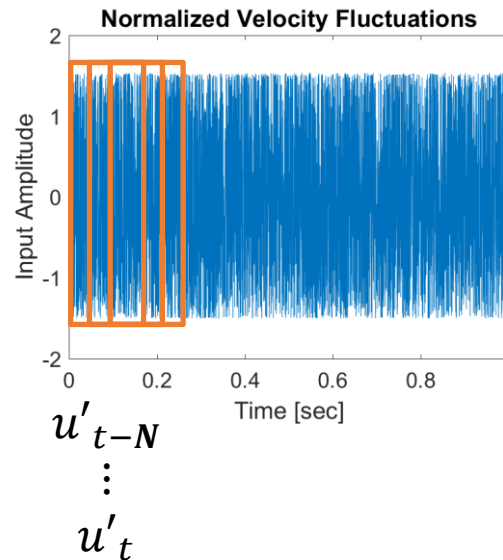
Similar steps as for Lorenz case except input and output are different signals

Overall exercise

- Objective: Develop a neural network model of the flame response
- 3 training datasets provided for different amplitudes of excitations
- Two approaches possible
 - Feedforward NN
 - Recurrent neural network
- Study of the accuracy of the trained NN depending on the training dataset used
- Study on the impact of the length of the dataset used for training

Steps for the exercise

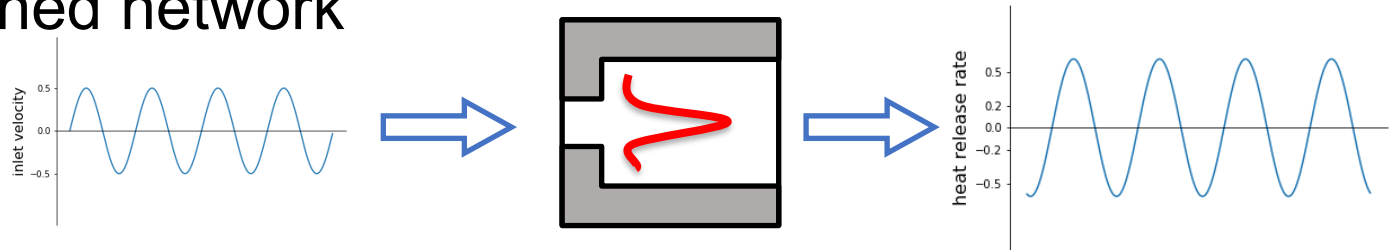
1. Preprocess the dataset
 - Consider undersampling
2. From the data provided, create the appropriate “input/output” pairs
 - Pay attention to your choice of “history” (N)
3. Design and train the network



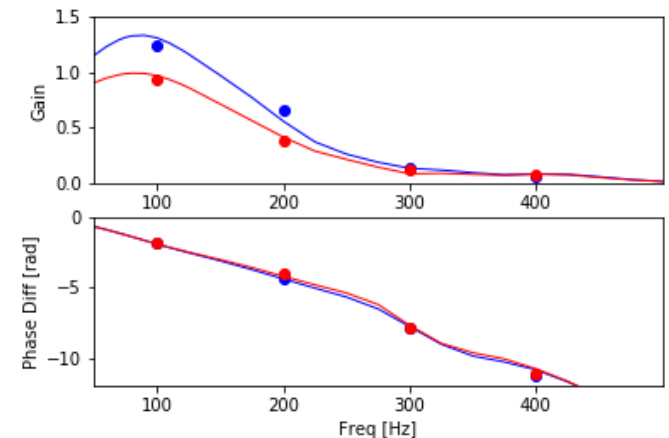
Assessment of the trained neural network : comparison with harmonic forcing

4. Compute the flame describing function $F(\omega, |u'|) = \frac{\dot{Q}'(\omega, |u'|)/\bar{\dot{Q}}}{u'(\omega, |u'|)/\bar{u}}$

- Create harmonic signals at given frequencies
- Feed them to your trained network

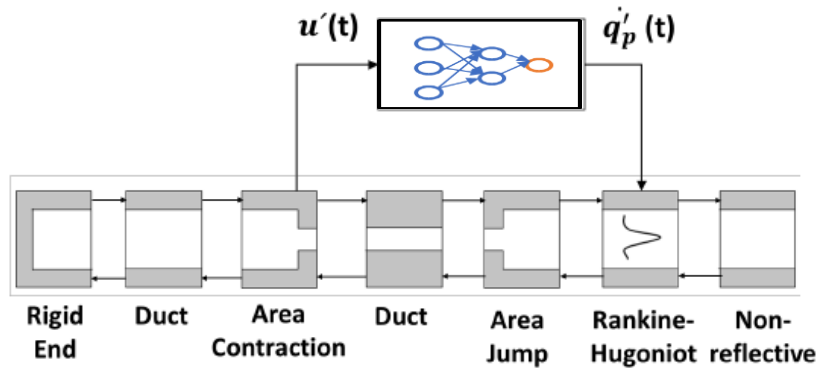


- Deduce the gain (“ratio” between input and output) and phase (normalised time shift between input/output)



Integrated NN and acoustic network predicts accurately the thermoacoustic instability

- Trained NN with acoustic model describes the gas turbine



Stability analysis for different designs possible

